

UAT Target Candidates

Status: research complete, TOML fixture stood up, remaining targets undecided. **Date:** 2026-08-09 **Purpose:** Select self-scoring conformance exercises for `drydock uat` beyond ReadingList (trivial UI) and CommonMark (mid-sized, flat spec).

All sizes and counts in this document were measured directly from the upstream repositories, not estimated.

Selection Criteria

A UAT target qualifies when it has:

- 1. **An external oracle.** The test suite is authored upstream, so passing is not self-graded.
- 2. **A fractional score.** Output is `N/M passing`, not binary, so the build loop has a gradient.
- 3. **Pure Python 3 + bash, no commercial or network dependency at test time.**
- 4. **Enough depth that one frontier-model pass is worse economics** than repeated scored Sonnet passes over decomposed stories.

Suite pre-partitioning is deliberately **not** a criterion. CommonMark's flat 652-example `spec.txt` offers the planner no seams and still worked; treating decomposability as a filter would only prove Drydock works on problems that were already carved up. It is a variable under test, and the candidate set below spans it on purpose.

Target	Where the difficulty lives	Suite pre-partitioned
TOML	Nowhere — breadth, not depth	Yes, by feature directory
jq	One place: generator / backtracking semantics	No — one flat 2,635-line file
HTML5	Spread across ~23 insertion modes	Yes, one file per mode
SQL engine	Everywhere, and unbounded without scoping	Partially

Note on the Toml Fixture's Run Harness Block

Not a rule for the repository — just what the Toml fixture does. `INSTRUCTIONS.md` carries a `## Run Harness` section holding the complete `full_test.sh` verbatim, an `ls sources/` step telling the builder to correct paths against reality rather than assume the layout, and a prohibition on any edit other than a path correction. `Definition of Done` points at that section instead of restating the script.

The path-verification step exists because `uat.py` copies fixture sources by basename into a flat `sources/` directory at the application root, so any subdirectory structure inside a fixture is lost.

Worth revisiting for later fixtures once there is evidence it changed the outcome. CommonMark and ReadingList are untouched.

Candidate Matrix

Bundle is what lands in `blueprint/sources/` and is paid for on every prompt. **Corpus** is test data on disk; it costs wall-clock, not tokens.

Target	Upstream	Stars	Bundle	$\approx$ tokens	Corpus	Cases	Harness	Verdict
TOML	<code>toml-lang/toml-test</code>	265	<code>specs/v1.0.0.md</code> — 25 KB	~6k	4.3 MB (embedded in binary)	709 at TOML 1.0 (210 valid / 499 invalid)	Go binary, stdin/stdout protocol	Built. Cheap bundle by 6x.
jq	<code>jqlang/jq</code>	35,417	<code>manual/dev/manual.yml</code> 144 KB + <code>parser.y</code> 22 KB + <code>lexer.l</code> 4.5 KB + <code>builtin.jq</code> 9.4 KB = 180 KB	~45k, sliceable to 15–25k	73 KB (4,097 lines)	~1,000	30-line Python runner over flat text	Flagship. Recommend second.
HTML5	<code>web-platform-tests/wpt</code> + <code>html5lib/html5lib-tests</code>	— / 260	WHATWG §13 extract — must be authored, est. 200–250 KB	~60k	2.3 MB	1,843 tree-construction + ~1,500 tokenizer	~80-line Python runner over <code>.dat</code> / <code>.test</code>	Strong, but needs a spec extractor first.
SQL (scoped)	<code>gregrahn/sqllogictest</code>	44	<code>about.wiki</code> format doc + explicit scope statement	~10k	3.0 MB (<code>select1-5</code> + <code>evidence</code>)	8,884 queries + 12 evidence files	C runner or <code>sqllogictest-rs</code>	Stretch. Scope must be a har Blueprint boundary.
SQL (full)	same	44	same	~10k	1.06 GB	605 files in <code>random/</code> + <code>index/</code>	same	Do not. Loop never terminates.
HTTP/2 server	<code>summerwind/h2spec</code>	731	RFC 9113 + RFC 7541	~90k	446 KB	146 named cases, grouped by RFC section	Go binary against your server port	Deferred. Different in kind — protocol state

Target	Upstream	Stars	Bundle	≈ tokens	Corpus	Cases	Harness	Verdict
								machines, not text.
JSON Schema	json-schema-org/JSON-Schema-Test-Suite	—	Draft 2020-12 core + validation	~70k	—	~5k, one file per keyword	Python runner	Deferred. Cleanest decomposition of any suite, but redundant with CommonMark kind.
YAML 1.2	yaml/yaml-test-suite	—	YAML 1.2 spec	large	—	~400	Python runner	Rejected. Same kind of exercise as CommonMark
Unicode UAX #29 / #14	UCD test files	—	UAX text	large	—	tens of thousands	Trivial	Rejected. Mechanically huge, no re-planning pressure.
OCI registry	opencontainers/distribution-spec	—	distribution spec	~50k	—	by workflow category	Go test binary against your HTTP endpoint	Deferred. Genuine server + pipeline flavor.
WebAssembly	WebAssembly/spec	—	core spec	very large	—	~20k assertions	.wast runner	Rejected. It is an interpreter — closest to a pure compute kernel.

Per-Target Notes

TOML — built, run first

The entire normative specification ships **inside the test repository** as `specs/v1.0.0.md`, so the source bundle is one 25 KB file. Harness ergonomics are the best of any candidate: the decoder reads TOML on stdin, writes tagged JSON on stdout, and exits non-zero on invalid input. Scoring is one command.

`toml-test test -run 'valid/string/*'` scopes the suite by feature directory, giving free story-level scores. Groups present:

- valid: `array bool comment datetime float inline-table integer key spec-1.0.0 string table`
- invalid: the same plus `control encoding local-date local-datetime local-time`

Measured baseline. Python's stdlib `tomllib` scores **208/210 valid, 499/499 invalid** against TOML 1.0 (failing only the two UTF-8 BOM cases). The exercise is therefore worthless unless `tomllib`, `tomli`, `toml`, and `tomlkit` are forbidden — the fixture instructions forbid them explicitly.

Expected effort: half a day. Its real job is to prove the Go-harness path end to end and then serve as a fast regression target.

jq — the flagship

The bundle looks expensive at 180 KB, but `manual.yml` is structured **per-builtin** YAML, so Drydock can inject only the entries a story touches. That satisfies the AGENTS.md rule to include only relevant specification sections without extra tooling, and drops the realistic per-story bundle to 15–25k tokens.

Including `parser.y` and `lexer.l` mirrors what CommonMark did with `cmark.py`; the grammar is only 972 lines. The corpus is trivially small — `tests/jq.test` is 52 KB — but semantically brutal. `reduce` / `foreach`, path expressions, and generator backtracking are where a naive implementation dies, and it dies as a score rather than a judgment call.

This is the target that exercises "story too hard, split it further" for real, because the suite offers no partition to hide behind.

Test files: `jq.test` (2,635 lines), `man.test` (998), `onig.test` (224), `uri.test` (92), `base64.test` (47), `optional.test` (12). Format is flat text: program / input / expected output lines, blank-line separated.

HTML5 — best decomposition, needs setup work first

Upstream moved. `html5lib/html5lib-tests` HEAD is literally the commit "*Tree construction tests have moved to WPT*". What remains there is tokenizer, encoding, and serializer only. The 1,843 tree-construction cases now live in `web-platform-tests/wpt`:

```
git clone --depth 1 --filter=blob:none --sparse https://github.com/web-platform-tests/wpt.git
cd wpt && git sparse-checkout set html/syntax/parsing # 1.2 MB, 61 .dat files
```

The only candidate with a genuine bundle problem: WHATWG HTML has no `spec.txt` equivalent, so §13.2 (tokenization, ~80 states) and §13.2.6 (tree construction, ~23 insertion modes) must be extracted from the living standard into markdown. Budget that as real setup work.

Payoff is 61 `.dat` files that map 1:1 onto insertion modes with a built-in difficulty gradient. `doctype01.dat` is an afternoon; `adoption01.dat` (adoption agency algorithm) and `foreign-fragment.dat` (66 cases) are where re-planning triggers. Largest files: `tests16.dat` (197), `processing-instructions.dat` (124), `tests1.dat` (112), `template.dat` (112), `tests19.dat` (103).

Exclude `namedEntities.test` from the bundle — 1.13 MB of generated table data, not specification.

SQL engine — viable only scoped

`test/random` alone is 649 MB across 391 files; `test/index` is 413 MB across 214. That is not a token problem, it is a non-termination problem.

Scoped proposal: `test/evidence/` (12 files, 52 KB, hand-written and feature-named — `slt_lang_aggfunc`, `in1`, `in2`, `slt_lang_createview`, `slt_lang_update`, `slt_lang_createtrigger`) as the epic, with `select1.test` (1,000 queries, 12,188 lines) as the acceptance gate. Exclude `random/` and `index/` in the Blueprint. Even inside the scope, `select4.test` is 2,832 queries over 48,300 lines — do not aim the loop at it early.

Second caveat, and it makes this a different experiment: the normative semantics are **not in the bundle**. There is no SQL specification in that repository — `about.wiki` documents the file format, not the language. The builder relies on the model's prior SQL knowledge. Worth running, worth knowing that is what is being run.

Go Toolchain

Two candidates (TOML, h2spec) use prebuilt Go test binaries as the oracle. Go ships **prebuilt** — nothing compiles, it is an untar. BSD licensed, no account, no registry.

The system `/usr/bin/go` is 1.18.1 from apt and is **too old**: `toml-test` requires ≥ 1.19 and fails to build on `fmt.Appendf`. Verified working install:

```
curl -sL https://go.dev/dl/go1.24.6.linux-amd64.tar.gz -o go.tgz # 75 MB
sudo tar -C /usr/local -xzf go.tgz
export PATH=/usr/local/go/bin:$PATH
```

Note that the builder never writes Go under any of these targets — Go binaries only grade Python. "Drydock builds a Go project" is a separate experiment none of these run.

Running the Toml Fixture

One-time setup. The system `/usr/bin/go` is 1.18.1 from apt and cannot build the harness — `toml-test` needs 1.19 or newer and fails on `fmt.Appendf`.

```
cd /mnt/c/Users/barlo/projects/drydock

curl -sL https://go.dev/dl/go1.24.6.linux-amd64.tar.gz -o /tmp/go.tgz
sudo rm -rf /usr/local/go && sudo tar -C /usr/local -xzf /tmp/go.tgz
export PATH=/usr/local/go/bin:$HOME/.local/bin:$PATH
go version # expect go1.24.6

sh tests/uat/ToMl/setup_harness.sh # installs to ~/.local/bin
toml-test version # expect v2.2.0
```

Add the `PATH` line to `~/.bashrc` so the harness stays resolvable in later shells. Everything after this point is offline.

Run:

```
drydock uat Toml
```

Inspect the result:

```
ls -lt uat/runs | head -1 # newest run directory
RUN=uat/runs/$(ls -lt uat/runs | head -1)
cat "$RUN/toml/result.json"
sed -n '1,40p' "$RUN/SUMMARY.md"
```

Score the build by hand, or re-score a group after the fact:

```
cd "$RUN/toml/build/toml"
sh full_test.sh # full suite, the scored run
sh sources/run_conformance.sh -run 'valid/string/*'
sh sources/run_conformance.sh -json
sh sources/run_conformance.sh -script # emit -skip flags for current failures
```

Rebuild the proof kit without rebuilding the project:

```
drydock uat --report
```

Recommended Order

1. **TOML** — smallest bundle, validates the Go harness path, fast regression target after.
2. **jq** — the flagship. Undecomposed suite plus one genuinely hard semantic core.
3. **HTML5** — after the WHATWG §13 extractor exists. Best decomposition story in the set.
4. **SQL** (**evidence** + **select1 only**) — stretch, with the exclusion written into the Blueprint.

Next Steps

1. Install Go 1.24 and the harness, then run `drydock uat Toml` for the first real pass. Nothing below the LLM is unverified; the build loop itself has not been exercised on this fixture.
2. Read the run's `full_test.sh`. Confirm it matches the Run Harness block and that the builder did not add skip flags or mask the exit code. If it did, that is a finding about the instructions, not about the parser.
3. Record the observed score, pass count, token usage, and wall-clock in this file so jq has a baseline to compare against.
4. Stand up the jq fixture once the manual-slicing question below is decided.
5. Author the WHATWG §13 extractor. It gates HTML5 and nothing else can start it.

Open Decisions

- jq: wire per-builtin `manual.yml` slicing into the source bundle for pass 1, or ship the whole manual to keep the comparison with CommonMark clean.
- Go toolchain: install permanently at `/usr/local/go`, or keep toolchains per-target inside the workspace.
- HTML5: whether to author the §13 extractor as a Drydock utility or a one-off script.